

PROJECTE 1	
Mòdul: M0616- Projecte de desenvolupament web	
Professor: Albert Guardiola	
Data límit:	ENTREGA1 – 23/9/26
	ENTREGA2 – 30/9/26
	ENTREGA3 – 7/10/26
Mètode d'entrega: Per mitjà del Clickedu de l'assignatura.	

PROJECTE 1 - Automatització, integració contínua i desplegament d'un projecte web

PART A. CONTEXT I EXPLICACIÓ GENERAL DEL PROJECTE

A.1. Context

En un projecte de desenvolupament web professional, GitHub no s'utilitza únicament per guardar el codi font.

Un repositori pot incorporar un conjunt de processos automatitzats que permeten:

- treballar amb branques i Pull Requests;
- revisar els canvis abans d'incorporar-los;
- executar tests automàticament;
- detectar errors abans de fer un merge;
- gestionar les dependències del projecte;
- analitzar possibles vulnerabilitats;
- desplegar automàticament una aplicació.

En aquesta pràctica assumiràs el rol d'una persona desenvolupadora que ha de convertir un projecte web convencional en un projecte amb un **flux de treball DevOps automatitzat mitjançant GitHub**.

Partiràs d'una aplicació web desenvolupada amb **HTML**, **CSS** i **JavaScript** i hauràs d'anar incorporant progressivament diferents mecanismes d'automatització.

A.2. Projecte web de partida

El projecte de partida és **DevTasks**, una aplicació web senzilla de gestió de tasques.

L'aplicació permet:

- crear tasques;
- marcar-les com a completes;
- eliminar-les;
- filtrar-les;
- consultar estadístiques;
- conservar les dades mitjançant `localStorage`.

El projecte està desenvolupat amb:

- HTML
- CSS
- JavaScript
- npm
- Vitest

Estructura inicial

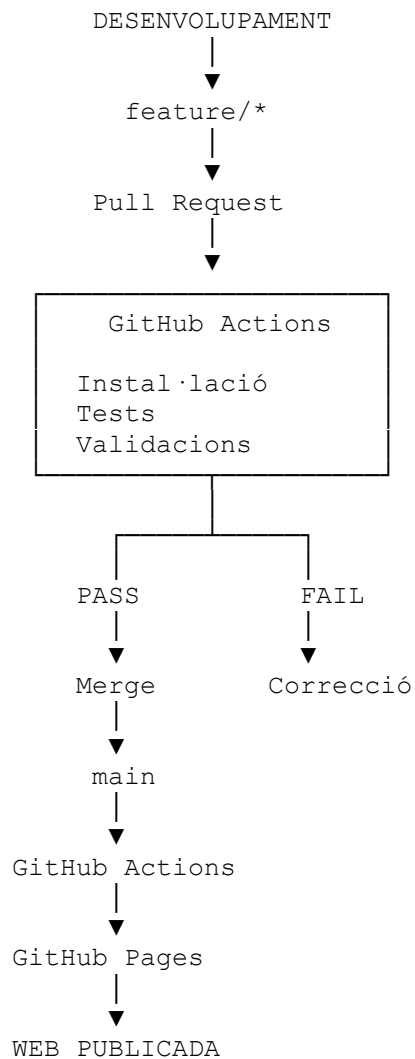
```
devtasks/  
├── index.html  
├── css/  
│   └── style.css  
├── js/  
│   └── app.js  
├── tests/  
│   └── app.test.js  
├── package.json  
├── .gitignore  
└── README.md
```

El projecte inicial conté també algunes funcions JavaScript susceptibles de ser separades i testades.

A.3. Objectiu general

L'objectiu de la pràctica és **transformar el projecte inicial en un repositori GitHub amb un flux de desenvolupament i desplegament automatitzat**.

Al final de la pràctica hauràs d'arribar a una arquitectura conceptual semblant a:



A més, el repositori haurà d'incorporar mecanismes de gestió de dependències i, opcionalment, mecanismes d'anàlisi de seguretat.

A.4. GitHub Skills

Abans de començar la configuració DevOps del projecte hauràs de completar els tutorials de GitHub Skills indicats a continuació.

Skills obligatoris

Bloc 1 — GitHub i col·laboració

1. **Introduction to GitHub**
2. **Review pull requests**
3. **Resolve merge conflicts**
4. **GitHub Pages**

Bloc 2 — Automatització

4. **Hello GitHub Actions**
5. **Test with Actions**
6. **Secure your repository's supply chain**

Skill d'ampliació

7. **Introduction to CodeQL**

El curs de CodeQL és opcional i forma part de les activitats d'ampliació.

A.5. Relació entre els Skills i el projecte

Els tutorials no són una activitat independent.

Cada Skill haurà de ser posteriorment aplicat al projecte web.

GitHub Skill	Coneixement adquirit	Aplicació al projecte
Introduction to GitHub	Repositoris, commits, branques	Crear i organitzar el repositori
Review pull requests	Pull Requests i revisió	Flux de treball basat en PR
GitHub Pages	Publicació web	Desplegament de l'aplicació
Hello GitHub Actions	Workflows, jobs i steps	Crear els pipelines
Test with Actions	CI i tests	Executar tests automàticament
Secure repository supply chain	Dependències i Dependabot	Automatitzar actualitzacions
Introduction to CodeQL	Code Scanning	Anàlisi de seguretat — ampliació

PART B. ENTREGUES 1 I 2 — EVIDÈNCIES GITHUB SKILLS

La primera part de la pràctica consisteix a demostrar que has realitzat els tutorials de GitHub Skills.

Les evidències es dividiran en dues entregues.

ENTREGA 1 — Skills de GitHub col·laboratiu

Tutorials

Hauràs de completar:

- Introduction to GitHub
- Review pull requests
- Resolve merge conflicts
- GitHub Pages

Evidències

Entrega un document PDF que contingui:

2. Evidència de finalització

Per cada tutorial hauràs d'incloure captures de pantalla que permetin comprovar que l'activitat ha estat completada.

3. Reflexió

Per cada Skill hauràs d'explicar breument:

- què has après;
- quin problema resol;
- com l'aplicaràs al projecte DevTasks.

No es tracta únicament d'explicar què fa l'eina. Has de relacionar el coneixement amb el projecte.

Exemple

SKILL: Review Pull Requests

He après a crear Pull Requests i revisar els canvis abans d'incorporar-los a la branca principal.

APLICACIÓ AL PROJECTE:

Utilitzaré Pull Requests per incorporar les modificacions realitzades en branques feature/* a main.

ENTREGA 2 — Skills DevOps

Tutorials

Hauràs de completar:

- Hello GitHub Actions
- Test with Actions
- Secure your repository's supply chain

Evidències

Entrega un segon document PDF amb:

1. Evidències de finalització

Captures de pantalla dels tutorials completats.

2. Skill adquirit

Per cada tutorial explica:

- què has après;
 - quin problema resol;
 - quin element del projecte hauràs de configurar gràcies a aquest coneixement.
-

PART C. APLICACIÓ DELS SKILLS AL PROJECTE WEB

A partir d'aquest punt hauràs d'utilitzar els coneixements adquirits als tutorials per transformar el projecte DevTasks.

La implementació s'ha de realitzar **pas a pas i en l'ordre indicat**.

PAS 1. Crear el repositori del projecte

Crea un repositori nou a GitHub per al projecte.

El repositori haurà de tenir:

- un nom identificatiu;
- un `README.md`;
- una branca principal `main`;
- el projecte web de partida.

Clona el repositori i incorpora-hi els fitxers inicials de DevTasks.

Evidència

Hauràs de poder demostrar:

- existència del repositori;
 - estructura de fitxers;
 - historial de commits.
-

PAS 2. Preparar el projecte JavaScript

Abans de començar amb GitHub Actions, prepara el projecte per poder executar tests localment.

1. Instal·la NodeJs al teu ordinador. El necessitem per fer servir el gestor de paquets `npm`:
2. Executa:

```
npm install
```

i comprova que els tests es poden executar amb:

```
npm test
```

El projecte inicial inclou funcions com:

```
createTask()  
isValidTask()  
filterTasks()  
getTaskStats()
```

Aquestes funcions representen la lògica de negoci de l'aplicació.

PAS 3. Refactoritzar el JavaScript

Abans d'automatitzar els tests hauràs de separar la lògica de negoci de la manipulació del DOM.

La versió inicial conté tota la funcionalitat a:

```
js/app.js
```

Hauràs de refactoritzar-la.

L'objectiu és arribar, com a mínim, a:

```
js/  
├─ app.js  
└─ taskManager.js
```

taskManager.js

Haurà de contenir la lògica que pugui ser executada i testada independentment del navegador.

Per exemple:

```
createTask()  
isValidTask()  
filterTasks()  
getTaskStats()
```

app.js

Haurà de ser responsable principalment de:

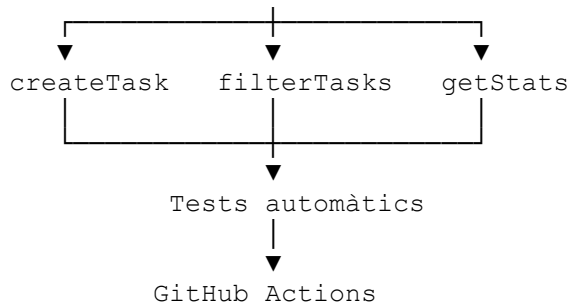
- interactuar amb el DOM;
- gestionar els events;
- actualitzar la interfície;
- utilitzar les funcions de `taskManager.js`.

Per què fem aquest refactor?

Perquè en un entorn DevOps no només interessa tenir tests, sinó disposar d'un codi que sigui **testable i mantenible**.

L'objectiu és aconseguir:

```
taskManager.js  
|
```



PAS 4. Crear i ampliar els tests

Crea tests per comprovar el comportament de les funcions de `taskManager.js`.

Hauràs de disposar d'un mínim de **5 tests significatius**.

Com a mínim hauràs de comprovar:

- validació d'una tasca;
- creació d'una tasca;
- filtratge de tasques;
- càlcul d'estadístiques;
- algun cas límit.

No es consideraran suficients tests que simplement comprovin que una funció existeix.

Els tests han de comprovar **comportaments reals**.

Executa els tests localment:

```
npm test
```

Abans de continuar, tots els tests han de passar.

En aquest moment encara estàs executant els tests localment. Posteriorment aprendràs a fer que GitHub els executi automàticament.

PAS 5. Crear una branca de funcionalitat

A partir d'aquest moment no hauràs de treballar directament sobre `main`.

Crea una branca amb un nom descriptiu, per exemple:

```
feature/refactor-javascript
```

Realitza el refactor i els tests en aquesta branca.

Fes commits petits i descriptius.

PAS 6. Crear una Pull Request

Quan hakis acabat la modificació:

1. Puja la branca a GitHub.
2. Crea una Pull Request cap a `main`.
3. Descriu els canvis realitzats.
4. Revisa els fitxers modificats.

La Pull Request serà el punt d'entrada del nostre procés d'Integració Contínua.

PAS 7. Crear el primer workflow de GitHub Actions

Ara hauràs d'automatitzar una tasca que fins ara feies manualment.

Crea:

```
.github/  
└─ workflows/  
    └─ ci.yml
```

El workflow haurà de:

1. executar-se quan es faci una Pull Request cap a `main`;
2. obtenir el codi del repositori;
3. preparar Node.js;
4. instal·lar les dependències;
5. executar els tests.

El resultat haurà de ser visible a GitHub Actions.

PAS 8. Integrar els tests al pipeline

Modifica el workflow perquè els tests siguin una part obligatòria del procés de CI.

El pipeline haurà de representar:

```
Pull Request  
↓  
Checkout  
↓  
Node.js  
↓
```

```
npm install
  ↓
npm test
  ↓
PASS / FAIL
```

Si els tests fallen, el workflow haurà de fallar.

PAS 9. Provocar un error deliberadament

Per demostrar que entens el funcionament del pipeline, hauràs de provocar un error controlat.

Per exemple:

1. Modifica temporalment una funció.
2. Crea una Pull Request.
3. Comprova que GitHub Actions detecta el problema.
4. Observa que els tests fallen.
5. Corregeix el problema.
6. Torna a executar el pipeline.
7. Comprova que els tests tornen a passar.

Evidència obligatòria

Hauràs de capturar:

```
PR
  ↓
GitHub Actions
  ↓
TEST FAILED
```

i posteriorment:

```
PR
  ↓
GitHub Actions
  ↓
TEST PASSED
```

Aquesta serà una de les evidències més importants de la pràctica.

PAS 10. Protegir la branca main

Configura el repositori perquè `main` no pugui rebre directament els canvis sense passar pel procés establert.

Com a mínim hauràs d'aconseguir:

- Pull Request obligatòria;
- workflow de CI obligatori;
- no permetre fer merge si el CI falla.

L'objectiu és implementar:

```
CI PASS
  ↓
MERGE PERMÈS

CI FAIL
  ↓
MERGE BLOQUEJAT
```

PAS 11. Configurar GitHub Pages

Publica l'aplicació mitjançant GitHub Pages.

L'aplicació haurà de poder consultar-se mitjançant una URL pública.

Comprova que:

- la pàgina inicial funciona;
 - CSS es carrega;
 - JavaScript funciona;
 - les funcionalitats de l'aplicació són operatives.
-

PAS 12. Automatitzar el desplegament

No volem haver de publicar manualment l'aplicació.

Crea un segon workflow:

```
.github/
├── workflows/
│   ├── ci.yml
│   └── deploy.yml
```

Aquest workflow haurà de permetre desplegar automàticament la versió de `main` a GitHub Pages.

El flux final serà:

```
Pull Request
  ↓
CI
  ↓
Tests
  ↓
```

```
PASS  
↓  
Merge  
↓  
main  
↓  
Deploy  
↓  
GitHub Pages
```

PAS 13. Gestionar les dependències

El projecte utilitza dependències npm.

Configura Dependabot perquè comprovi periòdicament les actualitzacions de les dependències.

El repositori haurà d'incloure:

```
.github/  
└─ dependabot.yml
```

La configuració haurà de contemplar les dependències del projecte i les dependències dels GitHub Actions utilitzats, quan sigui aplicable.

PAS 14. Comprovar Dependabot

Comprova que la configuració de Dependabot és visible i correcta.

Si durant el període de treball Dependabot genera una actualització, revisa-la i documenta el procés.

Si no es genera cap actualització durant la pràctica, serà suficient demostrar correctament la configuració.

PAS 15. Actualitzar el README

El README haurà de documentar el projecte i el seu sistema DevOps.

Ha d'incloure, com a mínim:

Descripció

Què fa DevTasks.

Instal·lació

Com instal·lar les dependències.

Tests

Com executar els tests.

GitHub Actions

Quins workflows existeixen i què fa cadascun.

Pull Requests

Quin flux de treball s'ha establert.

Deploy

Enllaç a la web publicada.

Dependències

Com es gestionen les actualitzacions.

Arquitectura

Explicació de la separació entre:

```
app.js
taskManager.js
tests/
```

PAS 16. Comprovar el flux DevOps complet

Abans de l'entrega final hauràs de demostrar el funcionament complet del sistema.

Realitza una petita modificació al projecte.

El procés haurà de ser:

1. Crear branca
- ↓
2. Modificar codi
- ↓
3. Commit
- ↓
4. Push
- ↓
5. Pull Request
- ↓
6. GitHub Actions
- ↓

7. Tests
 - ↓
 8. PASS
 - ↓
 9. Merge
 - ↓
 10. Deploy automàtic
 - ↓
 11. GitHub Pages actualitzat
-

PAS 17. Activitat d'ampliació: CodeQL

Completa el tutorial:

Introduction to CodeQL

i incorpora Code Scanning al projecte.

L'objectiu és que el repositori pugui detectar automàticament determinats patrons de vulnerabilitats en el codi.

Skill aplicat

Introduction to CodeQL

Aquesta activitat és opcional.

ENTREGA 3 — REPOSITORI DEVOPS I EVIDÈNCIES

L'entrega final haurà d'incloure:

1. Repositori GitHub

Enllaç al repositori final.

El repositori haurà de ser accessible per al professorat.

2. Aplicació publicada

Enllaç a la versió publicada amb GitHub Pages.

3. Evidències

Entrega un document PDF amb captures que demostrin:

Git

- repositori;
- branques;
- commits.

Pull Request

- una PR real;
- revisió;
- merge.

CI

- execució correcta;
- tests superats.

Error

- execució amb tests fallits;
- posterior correcció.

Branch protection

- configuració de protecció de `main`.

Deploy

- execució del workflow;
- aplicació publicada.

Dependabot

- configuració;
- actualització/PR si se n'ha generat alguna.

Aplicació

- web publicada i funcionant.

Opcional

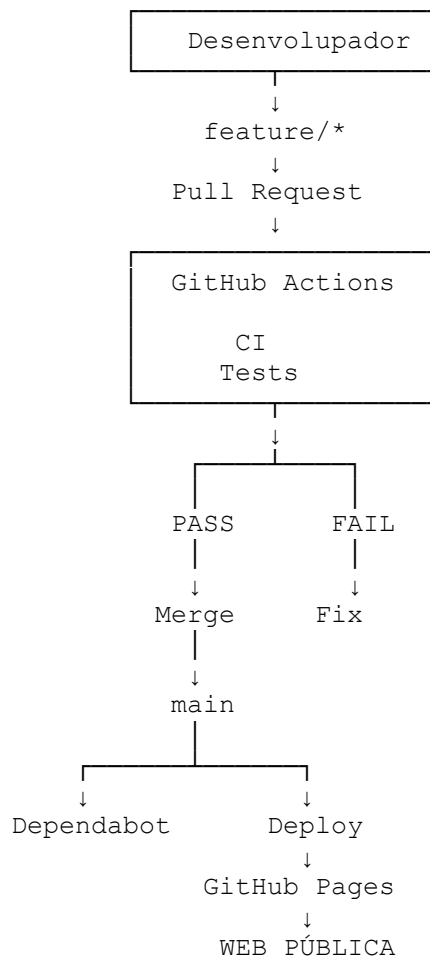
- CodeQL / Code Scanning.
-

RESULTAT FINAL

Al final de la pràctica hauràs transformat un projecte web convencional:

HTML
CSS
JavaScript

en un projecte amb un flux DevOps:



El producte final no és només l'aplicació DevTasks.

El producte final és el projecte web + el repositori + el flux de treball automatitzat + les evidències que demostren que funciona.